

AI-Generated Test Scripts for Web E2E Testing with ChatGPT and Copilot: A Preliminary Study

Maurizio Leotta
DIBRIS - Università di Genova
Italy
maurizio.leotta@unige.it

Filippo Ricca
DIBRIS - Università di Genova
Italy
filippo.ricca@unige.it

Hafiz Zeeshan Yousaf
DIBRIS - Università di Genova
Italy
s5130511@studenti.unige.it

Boni Garcia
Universidad Carlos III de Madrid
Spain
boni.garcia@uc3m.es

ABSTRACT

Automated testing is vital for ensuring the reliability of web applications. This paper presents a preliminary study on leveraging artificial intelligence (AI) models, specifically ChatGPT and Github Copilot, to generate test scripts for web end-to-end testing. Through experimentation, we evaluated the feasibility and effectiveness of AI language models in generating test scripts based on natural language descriptions of user interactions with web applications.

Our preliminary results show that AI-based generation generally provides an advantage over fully manual test scripts development. Starting from test cases clearly defined in Gherkin, a reduction in development time is always observable. In some cases, this reduction is statistically significant (e.g., Manual vs. a particular use of ChatGPT). These results are valid provided that the tester has some skills in manual test script development and is therefore able to modify the code produced by the AI-generation tools. This study contributes to the exploration of AI-driven solutions in web test scripts generation and lays the foundation for future research in this domain.

CCS CONCEPTS

• **Software and its engineering** → **Software verification and validation**; • **Computing methodologies** → Machine learning.

KEYWORDS

E2E Testing, Test Automation, Selenium WebDriver, GitHub Copilot, ChatGPT, LLM, Empirical Study.

ACM Reference Format:

Maurizio Leotta, Hafiz Zeeshan Yousaf, Filippo Ricca, and Boni Garcia. 2024. AI-Generated Test Scripts for Web E2E Testing with ChatGPT and Copilot: A Preliminary Study. In *28th International Conference on Evaluation and Assessment in Software Engineering (EASE 2024)*, June 18–21, 2024, Salerno, Italy. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3661167.3661192>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

EASE 2024, June 18–21, 2024, Salerno, Italy

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1701-7/24/06

<https://doi.org/10.1145/3661167.3661192>

1 INTRODUCTION

Automating end-to-end (E2E) tests is crucial to ensure the quality of web apps. Numerous approaches have been proposed for the automatic generation of test scripts (e.g., [2, 9, 10]), aiming to reduce the cost and time required for creating and executing E2E tests.

Advancements in Artificial Intelligence (AI), particularly in language models like GPT (Generative Pre-trained Transformer), have opened new perspectives for automatic test generation [8, 19]. Approaches like ChatGPT have exhibited remarkable capabilities in generating coherent and contextually relevant text across various, even complex, domains. Indeed, ChatGPT [1] has been successfully employed in a variety of contexts, including code writing and technical text generation. In the context of generating test scripts, ChatGPT offers the ability to create understandable and adaptable test scripts with minimal or no prior knowledge [8, 19]. Another example of how AI can be integrated into development environments to assist programmers in writing code is GitHub Copilot [6]. Copilot's ability to suggest code based on context provides an opportunity to explore its usage in generating test scripts.

Although scientific articles and empirical studies regarding ChatGPT and GitHub Copilot in the context of code generation are rapidly increasing [12, 18], to the best of our knowledge, works related to test script generation, especially for GUIs and web applications, are still few [20, 21]. However, both the scientific community and practitioners are questioning their utilization, especially regarding the potential to decrease the time required for developing test scripts, a notably expensive endeavor.

In this paper, we present a preliminary experiment aimed at comparing the effort required to manually develop web E2E test scripts with that needed when utilizing AI-based generation tools, namely: (1) ChatGPT and (2) GitHub Copilot. The goal is to evaluate potential benefits derived from the adoption of such AI-based tools.

The paper is organized as follows. Sect. 2 provides a basic background, allowing readers unfamiliar with GUI testing using AI-based tools to grasp the paper's content. Sect. 3 describes our preliminary empirical study with particular attention to the adopted procedure. Sect. 4 presents the preliminary results. Sect. 5 presents related works. Finally, Sect. 6 concludes the paper and presents some future directions.

2 E2E TESTING FOR WEB APPS

E2E testing is a critical aspect of software development, particularly for web applications. It involves testing an application's entire workflow, simulating real user scenarios from start to finish. E2E testing ensures that all components of the application function correctly together, providing confidence in its reliability. For our study, we used Selenium WebDriver¹ [4, 5, 14], one of the most widely used browser automation library. In our experiment, we utilized Java as the programming language and JUnit 5², a well-known unit testing framework, in conjunction with Selenium WebDriver to create test scripts. Selenium WebDriver allows testers to interact with web elements, perform actions like clicking buttons and entering text, while JUnit assertions are used to verify expected outcomes, all within automated test scripts. Furthermore, to describe the test cases, we have decided to use Gherkin [13], a popular language for writing concise and understandable test scenarios in a human-readable format. Gherkin allows us to define test cases in a structured manner, using keywords such as Given, When, Then, and And, facilitating clear communication between stakeholders and enabling easy translation into automated test scripts. Consider a simple login functionality test scenario described in Gherkin:

Feature: Login Functionality

As a user

I want to log in to the web application

So that I can access my account

Scenario: Successful Login

Given I am on the login page

When I enter my username "John" and password "pass"

And I click the login button

Then I should be redirected to the dashboard page

And I should see a welcome message "Hi John!"

In this example, Selenium WebDriver can be utilized to automate the actions described in the Gherkin scenario. The Selenium WebDriver script written in Java would navigate to the login page, locate the username and password input fields [11], enter the provided credentials, click the login button, and using JUnit assertions to verify that the user is redirected to the dashboard page and sees the expected welcome message.

2.1 Manual Development

Traditionally, E2E test scripts are developed manually by software testers. This manual process involves crafting test scripts that mimic user interactions with the web app, such as clicking buttons, entering text, and verifying expected outcomes. While effective, manual test script development can be time-consuming and prone to errors, especially for complex applications with numerous test cases.

2.2 ChatGPT

Developing E2E web test scripts with ChatGPT [1] involves providing natural language prompts describing the desired user interactions and expected outcomes. ChatGPT then generates code snippets based on these prompts, which can be further refined and integrated into an IDE, such as Visual Studio Code. By leveraging ChatGPT's language generation capabilities, developers can streamline the process of test script development and enhance the efficiency of E2E testing for web applications.

¹ <https://www.selenium.dev/documentation/webdriver/> ² <https://junit.org/junit5/>

How to develop E2E Web Test Scripts with ChatGPT: crafting E2E web test scripts with ChatGPT requires formulating natural language prompts that articulate the desired user interactions and anticipated outcomes. However, this can be done in different ways and with different approaches. We devised two approaches based on the capability of ChatGPT: **ChatGPT Lite** and **ChatGPT Max**.

ChatGPT Lite. With ChatGPT Lite, our approach focuses on expediting the creation of test scripts by providing natural language prompts corresponding to Gherkin code descriptions of desired test scenarios. At this point, the produced test script is copied as-is from the software tester into the IDE and entirely refined manually. This method prioritizes speed, but it may necessitate substantial manual refinement by the software tester.

ChatGPT Max. In contrast, our approach with ChatGPT Max aims for a more refined process. Starting with initial Gherkin code descriptions, the software tester engages in iterative refinement of the generated test scripts. She continues to request refinements until is satisfied with the code accuracy or no longer observes visible improvements. This method emphasizes precision and refinement, ensuring that the generated code meets desired specifications and maintains high-quality standards. Once copied into the IDE, the software tester should only make minor changes to the code (at least w.r.t. the ChatGPT Lite approach), namely the parts that ChatGPT is unable to correct.

2.3 Github Copilot

GitHub Copilot [6] is an AI-powered code completion tool developed by GitHub and OpenAI. Leveraging machine learning models, Copilot assists developers by suggesting code snippets and completing lines of code based on contextual understanding. By analyzing code patterns and documentation, Copilot aims to expedite software development and improve code quality through intelligent code generation and suggestions.

How to develop E2E Web Test Scripts with Copilot: developing E2E web test scripts with Copilot entails a seamless integration of this AI-powered tool into an Integrated Development Environment (IDE), such as Visual Studio Code. Within the familiar confines of the code editor, developers begin by providing natural language prompts or descriptions of the desired test scenarios (e.g., using Gherkin). These prompts serve as guiding cues for Copilot, which then analyzes the context and generates initial code suggestions. As Copilot generates code suggestions, developers engage in an iterative refinement process. This involves reviewing and enhancing the generated code to ensure alignment with the intended test scenarios. Developers may adjust variables, incorporate additional assertions, or refactor the code structure as needed. Through this iterative refinement, the web test scripts evolve to accurately represent the desired E2E testing scenarios. Once the scripts reach a satisfactory level of refinement, developers proceed to execute them against the web application under test. This execution phase serves to verify that the generated test script code works on the current web application. After this phase is finished the generated test scripts can be employed to verify the functionality of the next version of the application, validating that it behaves as expected under various user interactions.

3 EXPERIMENT

This section describes the experiment in a structured way, following the guidelines by Wohlin et al. [17].

The *goal* of the experiment is to compare the effort required to develop web E2E test scripts using different strategies: the classical manual development and using AI-based generation tools such as Copilot and ChatGPT. The experiment aims to evaluate the potential benefits of incorporating AI-based tools, with specific emphasis on assessing the overall development effort in terms of time required.

The results of the study can be interpreted from multiple *perspectives*: (a) Researchers, interested to empirically assess the benefits from adopting AI-generation tools for web test scripts development; (b) Practitioners, willing to understand whether such tools can provide a costs reduction when adopted in the industry.

The *context* of the study consists of a *participant* developing test scripts for eight different web applications (the *objects*).

Table 1 summarizes the main elements of the experiment. For replication purposes, the experimental package has been made available at <https://sepl.dibris.unige.it/AI-Gen-TS.php>.

3.1 Research Questions and Metrics

The research questions of the study are the following:

RQ1: Does the *effort* required to develop web E2E test scripts vary when using a manual approach or employing AI-assisted methods such as Copilot, ChatGPT Lite, and ChatGPT Max?

RQ2: Does the *effort* required to develop web E2E test scripts differ among three AI-based approaches: (a) Copilot vs. ChatGPT Lite, (b) Copilot vs. ChatGPT Max, and (c) ChatGPT Lite vs. ChatGPT Max?

To measure software development effort, software engineering researchers have discovered different metrics [15] such as LOC or time (e.g., Person-days, i.e., the time in days required for one person to complete a task). In this experiment we used time (self-reported by the Junior Tester) as a measure of effort since we believe it is the most appropriate in this context. Indeed LOC is not suitable because the final code developed should be very similar across all four treatments: thus LOC not allow capturing the effort required to develop or modify the code to achieve similar final results. Other metrics could have been evaluated for the AI-based treatments, such as the number of changes required to refine the code, but they would have significantly complicated the experimental procedure (for example, tracking every refinement request made to ChatGPT or Copilot). In order to make the development times required to create the various test scripts of different lengths comparable, we have defined the following metrics:

Goal	Comparing the effort required to develop web E2E test scripts using different strategies: manual development and using AI-based generation tools
Quality focus	Effort required to develop a test script
Context	Objects: 8 test suites developed with different tools Participant: 1 Junior Tester
Null hypothesis	No effect on Time (measured with the WDT metric)
Treatments	Manual, Copilot, ChatGPT Lite, ChatGPT Max
Dependent var	Time (measured with the WDT metric)

Table 1: Overview of the experiment

$TSDT$ = Test Script Development Time (in seconds)

$GLOC$ = number of Gherkin LOC describing the test script

Thus, the final derived metric used to measure the time required for developing the test scripts with the various treatments is the Weighted Development Time, computed as follows:

$$WDT = TSDT / GLOC \quad (\text{expressed in seconds per Gherkin LOC}).$$

3.2 Participant

This preliminary experiment involves only one participant, a Junior Tester with advanced skills in software testing, particularly in E2E web test script development. Specifically, he has expertise in developing test scripts based on Selenium WebDriver and JUnit 5 using Java. The participant lacked skills in using Copilot and ChatGPT for generating test scripts. As a result, we provided him with various materials on both AI-generation tools. Then, we asked the participant to develop four test suites using these tools to learn how to effectively utilize them for E2E test script development.

3.3 Test Suites (Objects)

The objects of the study are eight test suites for eight different web portals/applications, namely: LinkedIn³, Ebay⁴, Dropbox⁵, Trello⁶, Open Source Point of Sale⁷, PostIt⁸, Snappy⁹, ProShop eCommerce Platform¹⁰. The test suites directly exercise the online web portals in the first four objects, while in the second four objects, they work on web applications installed on a local web server. Each test suite is composed by 12 simple test scripts developed with the four considered treatments as explained in the following subsection.

3.4 Experimental Procedure

As a first step, we selected different web systems in order to diversify the characteristics of the test scripts to be developed. For this reason, we included both well-known web portals and some open source web applications. Together, they cover quite different kind of functionalities (e.g., ecommerce, chat app, cloud storage app, social media app).

Then, for each web application, one of the authors of this paper defined 12 Gherkin test cases to be implemented using the four treatments (i.e., Manual, Copilot, ChatGPT Lite, ChatGPT Max). For each application, we provided the Junior Tester the Gherkin test cases descriptions for each of the 12 test scripts to implement. The Gherkin descriptions of the test cases have been defined in detail, describing each interaction with the web elements of the various pages.

To reduce learning we avoided to ask the Junior Tester to develop the same test script with different treatments. The order of the treatments used to develop each test script in the eight developed test suites is reported in Table 2.

Thus, for each app the 12 test scripts were developed as follows: three test scripts with Manual, three with Copilot, three with ChatGPT Lite, three with ChatGPT Max. To balance the complexity of the developed test scripts over the four treatments we:

³ www.linkedin.com ⁴ www.ebay.com ⁵ www.dropbox.com ⁶ <http://trello.com>

⁷ opensourcepos.org

⁸ github.com/ihasham42/social-media-app

⁹ github.com/koolkishan/chat-app-react-nodejs

¹⁰ github.com/bradtraversy/proshop-v2

	App _n	App _{n+1}	App _{n+2}	App _{n+3}
Test Script _k	Manual	Copilot	ChatGPT Lite	ChatGPT Max
Test Script _{k+1}	Copilot	ChatGPT Lite	ChatGPT Max	Manual
Test Script _{k+2}	ChatGPT Lite	ChatGPT Max	Manual	Copilot
Test Script _{k+3}	ChatGPT Max	Manual	Copilot	ChatGPT Lite

Table 2: Treatment used for developing each test script of the various test suites: $n = \{1, 2\}$ (so 8 apps overall) and $k = \{1, 2, 3\}$ (so 12 test scripts per test suite overall)

- calculated the length of Gherkin test cases (defined as the number of steps);
- sorted the Gherkin test cases based on their length (for each app);
- alternate the development of the test scripts among the four treatments, as shown in Table 2.

During implementation of the test scripts, the Junior Tester employed GitHub Copilot version 1.172.0 (installed in Visual Studio Code version 1.87.0). In the case of ChatGPT, he employed the freely available version based on the gpt-3.5-turbo model (i.e., the one accessible via the online chat). For both ChatGPT and Copilot the first prompt used to generate each test script was "Generate an E2E Selenium Webdriver test script in Java for the provided Gherkin with suitable implicit or explicit waits: [Gherkin specifications]".

3.5 Variables and Hypotheses Formulation

Our experiment has one *independent variable* (also called "main factor" or *treatment variable*) with four possible levels: (1) *Manual*, (2) *Copilot*, (3) *ChatGPT Lite*, and (4) *ChatGPT Max*. Since there is only one factor with four levels, then in this experiment we have four treatments.

The experiment has one *dependent variable*, on which treatments are compared: the *Time* required to develop a test script (measured with the WDT metric).

We can state the null hypotheses for RQ1 in this schematic way:

$$\begin{aligned} H_{0MCP}: \text{Time (Manual)} &= \text{Time (Copilot)} \\ H_{0MLite}: \text{Time (Manual)} &= \text{Time (ChatGPT Lite)} \\ H_{0MMax}: \text{Time (Manual)} &= \text{Time (ChatGPT Max)} \end{aligned}$$

We can state the null hypotheses for RQ2 in this schematic way:

$$\begin{aligned} H_{0CPLite}: \text{Time (Copilot)} &= \text{Time (ChatGPT Lite)} \\ H_{0CPMax}: \text{Time (Copilot)} &= \text{Time (ChatGPT Max)} \\ H_{0LiteMax}: \text{Time (ChatGPT Lite)} &= \text{Time (ChatGPT Max)} \end{aligned}$$

Since we could not find any previous empirical evidence that points out a clear advantage of one treatment vs the other, we formulated each H_0 as non-directional hypotheses. The objective of a statistical analysis is to reject the null hypotheses above, so accepting the corresponding alternative ones, H_a .

3.6 Analysis Procedure

After computing descriptive statistics and showing the results with boxplots, we used the Wilcoxon test [16] to compare the effects of the treatments. In particular, for each test script, we calculated the Weighted Development Time *WDT* as described in Sect. 3.1.

In all the statistical tests, we decided, as it is customary, to accept a probability of 5% of committing Type-I-error (α) [17], i.e., rejecting the null hypothesis when it is actually true.

Treatment	Mean	Median	SD
Manual	57.471	53.666	22.679
Copilot	46.283	45.000	12.803
ChatGPT Lite	50.771	47.333	20.407
ChatGPT Max	41.058	38.376	10.496

Table 3: Time: descriptive statistics per Treatment.

Since we performed three different analyses on the *Time* dependent variable for each treatment (i.e., once a treatment is set, it is compared with each of the other three treatments), we cannot use $\alpha = 0.05$; we need to compensate for repeated statistical tests. While several techniques are available, we opted for the most conservative one, the Bonferroni correction [3]. In a nutshell, the conclusion will be taken by comparing the p-value to a corrected significance level $\alpha_B = \alpha/nt$, where nt is the number of statistical tests performed. Thus, in our case, $\alpha_B = 0.05/3 = 0.016$.

While the statistical tests allow checking the presence of significant differences, they do not provide any information about the magnitude of such differences. Therefore, we used the non-parametric Cliff's delta ($|d|$) effect size [7]. The effect size is considered small for $0.148 \leq |d| < 0.33$, medium for $0.33 \leq |d| < 0.474$ and large for $|d| \geq 0.474$.

4 RESULTS

To answer the two **RQ**, let us start with a short description of the results from the experiments analyzing the effect of the main factor on the dependent variable *Time*. In general, the development of all the 96 test scripts took a total of just above 11 hours (i.e., about 1.5 hours per test suite). The total length in Gherkin LOC of the test script descriptions is 1021 corresponding to, on average, 10.6 Gherkin LOCs per test script. Each Gherkin LOC has in general a similar implementation complexity (i.e., retrieving the web element to interact with with a specific locator and then perform a Web-Driver action), so the effort for implementing a test script can be considered proportional to the length of its Gherkin specification. Table 3 summarizes the essential descriptive statistics (i.e., median, mean, and standard deviation) of *Time* for each treatment computed over all the eight developed test suites (96 test scripts overall).

From the table, it is evident that Manual development took the higher time, with a median time of about 54 seconds per Gherkin LOC. In second and third place are the approaches based on ChatGPT Lite and Copilot with quite similar results (about 47 and 45 sec/LOC respectively). Finally, the approach that required the lower development time is ChatGPT Max with about 38 sec/LOC.

Fig. 1 compares the WDT of the test scripts developed with different treatments. Each box plot represents the distribution of 24 points since three test scripts per treatments were developed for each of the eight test suites.

4.1 RQ1: Manual vs. AI assisted approaches

RQ1a: Manual vs. Copilot. By applying a Wilcoxon test, we found that the difference in terms of time between the Manual and the Copilot treatments is statistically not significant (due to the Bonferroni correction), as $p\text{-value}=0.0431$.

RQ1b: Manual vs. ChatGPT Lite. By applying a Wilcoxon test, we found that the difference in terms of time between the Manual

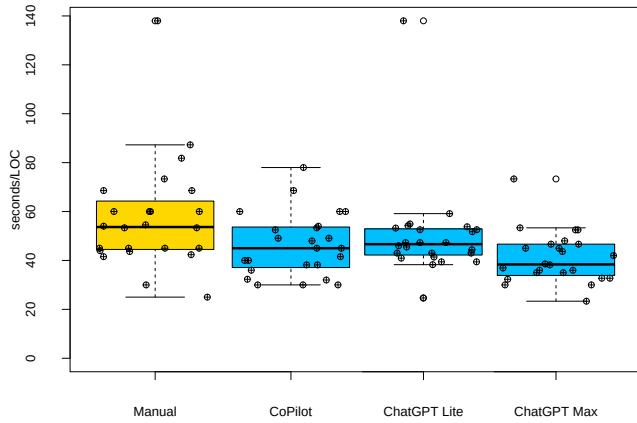


Figure 1: Boxplots comparing the WDT of the test scripts among different treatments (distribution of 24 points per treatment, i.e., 3 test scripts over 8 test suites).

and the ChatGPT Lite treatments is not statistically significant, as $p\text{-value}=0.1488$.

RQ1c: Manual vs. ChatGPT Max. By applying a Wilcoxon test, we found that the difference in terms of time between the Manual and the ChatGPT Max treatments is statistically significant, as $p\text{-value}=0.0008$. The effect size is large $d=+0.565$. Therefore, we can reject the null hypothesis H_{0MMax} and accept H_{aMMax} .

4.2 RQ2: Comparing AI assisted approaches

RQ2a: Copilot vs. ChatGPT Lite. By applying a Wilcoxon test, we found that the difference in terms of time between the Copilot and the ChatGPT Lite treatments is not statistically significant, as $p\text{-value}=0.3324$.

RQ2b: Copilot vs. ChatGPT Max. By applying a Wilcoxon test, we found that the difference in terms of time between the Copilot and the ChatGPT Max treatments is not statistically significant, as $p\text{-value}=0.1459$.

RQ2c: ChatGPT Lite vs. ChatGPT Max. By applying a Wilcoxon test, we found that the difference in terms of time between the ChatGPT Lite and the ChatGPT Max treatments is statistically significant, as $p\text{-value}=0.0045$. The effect size is large $d=+0.479$. Therefore, we can reject the null hypothesis $H_{0LiteMax}$ and accept $H_{aLiteMax}$.

4.3 Discussion

From the results obtained, it is evident that employing an AI-based generation tool for creating test scripts provides an advantage in terms of reducing the time required for the development. In some cases, the advantage is statistically significant, but a trend in the same direction is present across all treatments. The results have shown that ChatGPT Lite is the AI-based approach that requires the most time: this is reasonable as once the Gherkin is provided to ChatGPT and the tool generates the first test suite draft, the tester copies the result onto their development IDE and modifies the test to make it work. Although the tests were generally complete with all steps (as they were present in the detailed Gherkin we defined), many manual changes were necessary because ChatGPT lacks real knowledge about the structure of the tested web application. For example, locators used to locate the elements the test should interact

with are defined with plausible but incorrect strings. With Copilot, an improvement is observed, probably because it is executed within the IDE and thus has additional contextual information compared to ChatGPT, which it can leverage to generate more correct code (e.g., regarding specific test script settings). Finally, ChatGPT Max has proven to be the fastest treatment for test script development. Being able to suggest changes to the AI-based tool has allowed to quickly converge towards almost optimal solutions, which therefore require few manual modifications in the IDE. Clearly, these results are valid in the case of a tester who is proficient in the programming language used to code the test scripts and the libraries and frameworks used for test automation. Only in this way, it is reasonable to think that she is able to quickly modify the test scripts generated by an AI-based tool or even provide specific modification requests as required in the ChatGPT Max treatment.

Summary. This preliminary experiment shows that AI-based generation tools generally provide an advantage over fully manual development. Starting from test cases clearly defined in Gherkin, a reduction in development time is always observable. In some cases, this reduction is statistically significant (e.g., Manual vs. ChatGPT Max and ChatGPT Lite vs ChatGPT Max). These results are valid provided that the tester has some skills in manual test script development and is therefore able to effortlessly modify the code produced by the AI-generation tools.

4.4 Threats to Validity

This section discusses briefly the threats to validity that could affect our results: *internal*, *conclusion*, *construct* and *external* [17].

Internal validity threats concern factors that may affect a dependent variable (in our case, *Time*). Since the Junior Tester had to develop test scripts a learning effect may intervene. Concerning the learning on the specific treatment the Junior tester was previously trained and we asked him to develop four additional test suites following the same procedure employed in the experiment. This allowed for standardizing the Junior Tester skills across the four treatments. Moreover, to minimize any additional learning effect we devised the procedure described in Sect. 3.3 that alternate the development order among treatments. This procedure allowed to minimize the learning both within each individual test suite but also across different test suites. The fatigue effect is not applicable in our case since the tester had the possibility to develop the test suites over the course of several weeks.

Threats to *conclusion validity* refer to the extent to which conclusions drawn from the study are warranted by the data and the analysis conducted. They can be due to the sample size of the experiment (in our case, 96 test scripts developed by one Junior tester) that may limit the capability of statistical tests to reveal any effect, and the object size (eight test suites for eight different web apps), that could be insufficient to significantly reveal any effect. However, in several cases the Wilcoxon test provided sharp results and the difference between the treatments is in some cases relevant. Clearly, further experiments are needed to confirm the results obtained.

Threats to *construct validity* concern about whether the measures used in the study accurately represent the theoretical constructs being studied. Relying solely on time as a measure of effort may oversimplify the construct. Effort can encompass various components such as cognitive engagement, problem-solving strategies,

and perseverance, which may not be fully captured by time alone. However, using time as a measure of effort in software development experiments is a common practice in empirical SE research. Threats to *external validity* refer to factors that could affect the generalizability of the study's findings beyond the specific conditions under which our experiment was conducted. These could stem from the fact that our experiment involved one Tester, and the usage procedures of ChatGPT and GitHub Copilot might vary in real-world business practices. Clearly, further experiments with different test suites, more refined procedures and additional software testers are needed to confirm or contrast the obtained results.

5 RELATED WORK

The two most related works to ours are those of Zimmermann and Koziolok [20, 21]. The first paper [20] presents a novel method for GUI-based software testing, leveraging GPT-3. It interprets natural language test cases and navigates applications programmatically, overcoming memory limitations by integrating GUI element states into input prompts. The authors recommend using a test automation framework and provide a tool for simplified training data acquisition. To demonstrate the feasibility of the proposed approach, a sample application featuring a grid of 25 buttons for testing with GPT-3 was developed. The application tasked the GPT-3 agent with clicking the buttons in a specific order according to the corresponding test case description. This approach aims to enhance testing efficiency by eliminating manual input and enabling non-technical users to input test cases for desktop and mobile apps seamlessly.

The second paper [21], on the same line as the previous one, introduces a method for GUI testing in web applications by integrating GPT-4 with Selenium WebDriver. GPT-4's pre-training grants it robust inference capabilities, reducing the need for extensive training data and human testers. The study compares this integrated approach with monkey testing and demonstrates its superiority in achieving branch coverage. Although related, ours is an empirical study rather than a method proposal like these two works, and the goal is completely different. In our case, we aim to investigate whether these new AI-based tools can be useful in speeding up the production of test scripts.

6 CONCLUSION AND FUTURE WORK

In this paper, we have presented a preliminary experiment to compare the effort required for manually developing web E2E test scripts with the effort needed when using AI-based generation tools, specifically Copilot and ChatGPT. The aim was to assess the potential benefits of adopting AI-based tools for test script development. The initial findings indicate that AI-based generation generally offers advantages over fully manual test script development. When starting from test cases defined in Gherkin, a consistent reduction in development time was observed. In some instances, this reduction was statistically significant, particularly when comparing manual development to a specific use of ChatGPT (i.e., Max). However, it is important to note that these benefits rely on Software Testers possessing skills in manual test script development. This expertise enables them to effectively and rapidly modify the code generated by AI tools or provide the correct requests. Overall, these results

suggest promising potential for AI-based tools in accelerating and streamlining the development of web E2E test scripts.

In future work, we aim to conduct more extensive experiments with a diverse pool of participants, including both experienced software testers and graduate students. This broader approach will allow us to validate our initial preliminary findings and assess how varying skill levels impact the results.

Acknowledgement. This study was partially carried out within the “EndGame - Improving End-to-End Testing of Web and Mobile Apps through Gamification” project (2022PCCMLF) – funded by European Union – Next Generation EU within the PRIN 2022 program (D.D.104 - 02/02/2022 Ministero dell'Università e della Ricerca). This manuscript reflects only the authors' views and opinions and the Ministry cannot be considered responsible for them.

REFERENCES

- [1] Open AI. 2022. ChatGPT. <https://chat.openai.com/>. Accessed: 06 March 2024.
- [2] Diego Clerissi, Maurizio Leotta, Gianna Reggio, and Filippo Ricca. 2017. Towards the Generation of End-to-End Web Test Scripts from Requirements Specifications. In *Proceedings of 25th IEEE International Requirements Engineering Conference Workshops (REW 2017)*. IEEE, 343–350. <https://doi.org/10.1109/REW.2017.39>
- [3] Jean Dunn and Olive Jean Dunn. 1961. Multiple Comparisons Among Means. *ASA* (1961), 52–64.
- [4] Boni Garcia, Filippo Ricca, Jose M. del Alamo, and Maurizio Leotta. 2023. Enhancing Web Applications Observability through Instrumented Automated Browsers. *Journal of Systems and Software* 203 (2023). <https://doi.org/10.1016/j.jss.2023.111723>
- [5] Boni Garcia. 2022. *Hands-On Selenium WebDriver with Java*. O'Reilly Media, Inc.
- [6] Inc. GitHub. 2024. GitHub Copilot. <https://copilot.github.com/>. Accessed: 06 March 2024.
- [7] Robert J. Grissom and John J. Kim. 2005. *Effect sizes for research: A broad practical approach* (2nd ed.). Lawrence Earlbaum Associates.
- [8] Vitor Guilherme and Auri Vincenzi. 2023. An initial investigation of ChatGPT unit test generation capability. In *Proceedings of the 8th Brazilian Symposium on Systematic and Automated Software Testing (SAST '23)*. ACM, 15–24.
- [9] Maurizio Leotta, Filippo Ricca, Alessandro Marchetto, and Dario Olinas. 2024. An Empirical Study to Compare Three Web Test Automation Approaches: NLP-based, Programmable, and Capture & Replay. *Journal of Software: Evolution and Process (JSEP)* 36, 5 (2024), e2606. <https://doi.org/10.1002/smr.2606>
- [10] Ali Mesbah, Arie van Deursen, and Danny Roest. 2012. Invariant-Based Automatic Testing of Modern Web Applications. *IEEE Transactions on Software Engineering* 38, 1 (2012), 35–53. <https://doi.org/10.1109/TSE.2011.28>
- [11] Michel Nass, Emil Alégroth, Robert Feldt, Maurizio Leotta, and Filippo Ricca. 2023. Similarity-based Web Element Localization for Robust Test Automation. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 32, 3, Article 75 (2023), 30 pages. <https://doi.org/10.1145/3571855>
- [12] Fardin Ahsan Sakib, Saadat Hasan Khan, and A. H. M. Rezaul Karim. 2023. Extending the Frontier of ChatGPT: Code Generation and Debugging. *arXiv:2307.08260* [cs.SE]
- [13] SmartBear Software. 2019. Gherkin Reference. <https://cucumber.io/docs/gherkin/reference/>. Accessed: 06 March 2024.
- [14] Simon Stewart and David Burns. 2022. WebDriver, W3C Working Draft. <https://www.w3.org/TR/webdriver/>. [Online; accessed 1-April-2022].
- [15] Yuqing Wang, Mika V. Mäntylä, Zihao Liu, and Jouni Markkula. 2022. Test automation maturity improves product quality—Quantitative study of open source projects using continuous integration. *JSS* 188 (2022), 111259.
- [16] Wikipedia. [n. d.]. Mann-Whitney U test. Last access July 2021. Available at: https://en.wikipedia.org/wiki/Mann-Whitney_U_test.
- [17] C. Wohlin, P. Runeson, M. Höst, M.C. Ohlsson, B. Regnell, and A. Wesslén. 2000. *Experimentation in Software Engineering - An Introduction*. Kluwer Academic Publishers.
- [18] Burak Yetistiren, Isik Ozsoy, and Eray Tuzun. 2022. Assessing the quality of GitHub copilot's code generation. In *PROMISE 2022* (Singapore). ACM, 62–71.
- [19] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation. *arXiv:2305.04207* [cs.SE]
- [20] Daniel Zimmermann and Anne Koziolok. 2023. Automating GUI-based Software Testing with GPT-3. In *ICSTW 2023*. 62–65.
- [21] D. Zimmermann and A. Koziolok. 2023. GUI-Based Software Testing: An Automated Approach Using GPT-4 and Selenium WebDriver. In *ASEW 2023*. IEEE, Los Alamitos, CA, USA, 171–174.